

```
print("Welcome")
```

Welcome

Upload Your File

```
from google.colab import files  
  
uploaded = files.upload()
```

No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving table6_2026_04.csv to table6_2026_04.csv

Create Data frame

```
import pandas as pd  
  
df = pd.read_csv("table6_2026_04.csv")  
  
print(df.shape)  
# verify  
df.head()
```

[Show hidden output](#)

```
import numpy as np
```

Derived Features : Placeholders| Do not run

```
financial_progress_pct = (  
    cumulative_expenditure_cr / original_cost_cr  
) * 100  
  
progress_gap = (  
    financial_progress_pct - physical_progress_pct  
)  
  
start_delay_months = (  
    revised_start_date - approval_start_date  
) .days / 30.44
```

Cost Overrun Features

Verify Data First

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1981 entries, 0 to 1980
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   reporting_month                      1981 non-null   object
1   ministry                            1981 non-null   object
2   sector                              1981 non-null   object
3   sl_no                               1981 non-null   int64
4   project_name                        1981 non-null   object
5   agency                              1981 non-null   object
6   project_code                        1981 non-null   int64
7   legacy_ocms_code                    1184 non-null   object
8   pmgid                               1213 non-null   float64
9   state                               1981 non-null   object
10  approval_start_date                 1970 non-null   datetime64[ns]
11  revised_start_date                  1981 non-null   datetime64[ns]
12  target_doc                           1981 non-null   object
13  revised_doc                          1627 non-null   object
14  original_cost_cr                    1981 non-null   float64
15  revised_cost_cr                     1981 non-null   float64
16  cumulative_expenditure_cr           1981 non-null   float64
17  physical_progress_pct                1981 non-null   float64
18  cost_overrun                         1981 non-null   int64
19  financial_progress_pct               1981 non-null   float64
20  progress_gap                         1981 non-null   float64
21  start_delay_months                  1970 non-null   float64
dtypes: datetime64[ns](2), float64(8), int64(3), object(9)
memory usage: 340.6+ KB
```

df.head()

```
df.isna().sum()
```

[Show hidden output](#)

Revised Cost overrun experiment

```
df["cost_overrun"] = (  
    df["revised_cost_cr"] > df["original_cost_cr"]  
)<div data-bbox="164 765 431 833" data-label="Text">

```
cost_overrun
0 1439
1 542
Name: count, dtype: int64
```


```

Feature Engineering

```
cost_features = [  
    "ministry",  
    "sector",
```

```
    "agency",
    "state",
    "original_cost_cr",
    "cumulative_expenditure_cr",
    "physical_progress_pct",
    "financial_progress_pct",
    "progress_gap",
    "start_delay_months"
]

target = "cost_overrun"
```

Add them into Data Frame

```
financial_progress_pct = (
    cumulative_expenditure_cr / original_cost_cr
) * 100

progress_gap = (
    financial_progress_pct - physical_progress_pct
)

start_delay_months = (
    revised_start_date - approval_start_date
).days / 30.44
```

Run This

```
# 1. Financial progress
df["financial_progress_pct"] = (
    df["cumulative_expenditure_cr"]
    / df["original_cost_cr"].replace(0, np.nan)
) * 100
```

```
# 2. Progress gap
df["progress_gap"] = (
    df["financial_progress_pct"]
    - df["physical_progress_pct"]
)
```

Clean the date time

```
df["approval_start_date"] = pd.to_datetime(
    df["approval_start_date"], format="%m/%Y", errors="coerce"
)

df["revised_start_date"] = pd.to_datetime(
    df["revised_start_date"], format="%m/%Y", errors="coerce"
)
```

```
# 3. Start delay in months
df["start_delay_months"] = (
    (df["revised_start_date"].dt.year -
     df["approval_start_date"].dt.year) * 12
    +
    (df["revised_start_date"].dt.month -
     df["approval_start_date"].dt.month)
)
```

Check Null values for Features

```
print(df[cost_features].isna().sum())
```

```
ministry          0
sector            0
agency            0
state             0
original_cost_cr  0
cumulative_expenditure_cr  0
physical_progress_pct  0
financial_progress_pct  0
progress_gap      0
start_delay_months 11
dtype: int64
```

✓ Convert dates

```
df["approval_start_date"] = pd.to_datetime(
    df["approval_start_date"],
    format="%m/%Y",
    errors="coerce"
)
```

```
df["revised_start_date"] = pd.to_datetime(
    df["revised_start_date"],
    format="%m/%Y",
    errors="coerce"
)
```

Convert String to Numeric values

```
# Make physical progress numeric
df["physical_progress_pct"] = pd.to_numeric(
    df["physical_progress_pct"],
    errors="coerce"
)
```



Prepare X and Y

```
X = df[cost_features]
y = df["cost_overrun"]
```

prepare train/test split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
X_train: (1584, 10)
X_test  : (397, 10)
y_train: (1584,)
y_test  : (397,)
```

Define Numerical and categorical columns

```
categorical_features = [
    "ministry",
    "sector",
    "agency",
    "state"
]

numeric_features = [
    "original_cost_cr",
    "cumulative_expenditure_cr",
    "physical_progress_pct",
    "financial_progress_pct",
    "progress_gap",
    "start_delay_months"
]
```



Preprocessing pipeline

Import requirements

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
```

Now Convert

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

Create Preprocessor

```
preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])
print("done")
```

done

Create Model : Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

cost_model = Pipeline([
    ("preprocessing", preprocessor),
    ("model", LogisticRegression(
        class_weight="balanced",
        max_iter=1000
    ))
])
```

✓ Train Model

```
cost_model.fit(X_train, y_train)

print("Cost Overrun Model trained successfully.")
```

Cost Overrun Model trained successfully.

Prediction Checking

```
y_pred = cost_model.predict(X_test)

y_prob = cost_model.predict_proba(X_test)[: , 1]

print(y_pred[:10])
print(y_prob[:10])
```

```
[0 0 1 0 0 0 0 0 1 0]
[0.36937314 0.49377837 0.52575053 0.02486602 0.1652818  0.07039249
 0.27811489 0.08392131 0.94461637 0.05187198]
```

Model evaluation

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("ROC-AUC:")
print(roc_auc_score(y_test, y_prob))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

         0       0.90      0.75      0.82         288
         1       0.54      0.77      0.63         109

   accuracy              0.76         397
  macro avg              0.72         397
 weighted avg              0.80         397

ROC-AUC:
0.8536888379204893
Confusion Matrix:
[[216  72]
 [ 25  84]]
```

✓ Model Ready to Save

```
import joblib

joblib.dump(cost_model, "cost_overrun_model_v1.pkl")

print("Model saved successfully.")
```


Model saved successfully.

✓ Download Model Weight

```
from google.colab import files  
  
files.download("cost_overrun_model_v1.pkl")
```